

PTT TALK / OPERATOR EDITION

Deployment, Build, and Verification Guide

A complete runbook for private-team deployment on K3s or Cloudflare, mobile signing, encrypted voice validation, operations, and evidence-driven release handoff.

PRODUCT BASELINE PTT Talk 0.1.28 (31)

PROTOCOL 1.1

PRIMARY TARGET Single-tenant K3s + Helm 3

ALTERNATE TARGET Cloudflare managed edge

DOCUMENT VERSION 1.0 / 03 September 2026

Ship only when the evidence says it works. A green build is not proof of audible device-to-device voice.

Contents

- 1. What is being deployed 4**
 - 1.1 Deployment choices 4
- 2. Automation safety contract 4**
 - 2.1 Destructive-action stop points 5
- 3. Repository map 5**
- 4. Inputs that must exist before deployment 6**
 - 4.1 Common inputs 6
 - 4.2 K3s inputs 6
 - 4.3 Cloudflare inputs 6
 - 4.4 Mobile release inputs 6
- 5. Establish a clean, reproducible baseline 7**
- 6. Validate source before packaging 7**
- 7. Build immutable K3s application images 8**
- 8. Deploy on K3s with Helm 8**
 - 8.1 Confirm the target 8
 - 8.2 Configure DNS, TLS, and firewall 8
 - 8.3 Create the protected values file 8
 - 8.4 Render, inspect, and install 10
 - 8.5 External service verification 11
 - 8.6 Bootstrap the first administrator 11
 - 8.7 Confirm email and push delivery 11
- 9. Deploy the Cloudflare implementation 12**
 - 9.1 Create an operator overlay 12
 - 9.2 Authenticate and validate locally 12
 - 9.3 Provision staging resources 12
 - 9.4 Provision and deploy production 12
 - 9.5 Cloudflare backup and rollback 13
- 10. Custom domains, application IDs, and verified links 13**
- 11. Build and install Android 14**
 - 11.1 Toolchain 14

- 11.2 Debug and emulator build 14
 - 11.3 Signed Play bundle 14
- 12. Build and install iOS 14
 - 12.1 Fetch pinned libsignal 14
 - 12.2 Build simulator dependencies and app 14
 - 12.3 Signed TestFlight IPA 14
- 13. Functional and security test matrix 15
 - 13.1 Automated local and CI gates 15
 - 13.2 Simulator tests 15
 - 13.3 Deployed two-client media automation 16
 - 13.4 Physical-device voice release gates 16
 - 13.5 Long-duration and lifecycle tests 17
 - 13.6 Security gate 17
- 14. Release-gate verification 17
- 15. Backup, restore, upgrade, and rollback operations 18
 - 15.1 K3s backup drill 18
 - 15.2 K3s upgrade 18
 - 15.3 Secret rotation 18
- 16. Monitoring and incident readiness 19
- 17. Troubleshooting 20
- 18. Production go/no-go checklist 20
- 19. AI-agent execution protocol 21
 - 19.1 Required evidence manifest 21
- 20. Related source documents 22

This is the source-of-truth runbook for compiling, installing, operating, and testing PTT Talk from a clean checkout. It is intentionally explicit. An automation agent may follow it, but it must obey the safety contract below and must never interpret a successful build as proof that real encrypted voice works on physical devices.

1. What is being deployed

PTT Talk is a private-team, end-to-end encrypted push-to-talk and collaboration application. The current beta includes:

- Android and iOS clients with encrypted one-to-one and channel voice.
- Authenticated floor control, SOS priority, presence, recents, encrypted history, and safety numbers.
- End-to-end encrypted chat with files, recorded voice, and video attachments.
- Invitation-based enrollment, two independently keyed devices per account, device linking, recovery, and revocation.
- A responsive administrator console for invitations, members, devices, channels, roles, retention, audit events, integrations, and operational health.
- Push-assisted wake through FCM and separate APNs production and sandbox credentials.
- A UDP relay on K3s with automatic encrypted WebSocket/TLS media fallback.
- A Cloudflare implementation whose media path is encrypted WebSocket/TLS only.

The server stores account and delivery metadata. Message bodies, attachments, history audio, and media frames are ciphertext. There is no plaintext media fallback.

1.1 Deployment choices

Requirement	K3s + Helm	Cloudflare edge
Status	Supported self-hosted target	Supported alternate implementation
Compute	Control, relay, and admin containers	Worker and Durable Objects
Durable records	PostgreSQL	D1
Floor/presence	Redis	Durable Objects and D1
Ciphertext objects	MinIO/S3-compatible store	R2
Media	Authenticated UDP; encrypted TLS fallback	Encrypted WebSocket/TLS
Delivery jobs	Control-service workers	Queues with dead-letter queues
Best fit	Teams controlling a K3s host and network	Teams preferring managed edge services

Do not combine the two back ends in one production instance. Select one authoritative instance URL and one data store set.

2. Automation safety contract

An AI agent executing this guide MUST:

1. Work from a named commit and record `git rev-parse HEAD` before building.
2. Refuse to continue with unresolved placeholders, a dirty source tree it did not create, or an unexpected Git remote.
3. Keep production values, private keys, tokens, service-account JSON, signing files, and exported databases outside the repository.
4. Never print secrets, put secrets in command-line arguments, commit them, upload them as CI artifacts, or include them in support reports.

5. Use interactive secret input or protected files with mode `0600`.
6. Build application images from the selected commit and deploy immutable image digests. The example application tags in `deploy/helm/ptt/values.yaml` are documentation defaults, not published release artifacts.
7. Render and inspect infrastructure changes before applying them.
8. Take and verify a coordinated backup before any production upgrade or schema change.
9. Never restore, delete, uninstall, rotate active credentials, revoke devices, change DNS, or publish mobile builds without explicit operator authorization for the exact target.
10. Stop if a health check, migration, backup verification, cryptographic contract test, entitlement check, or physical-device gate fails. It must not bypass a failing gate.
11. Redact email addresses, account identifiers, device credentials, keys, tokens, ciphertext, and audio from evidence bundles.
12. Report `PASS`, `FAIL`, `BLOCKED`, or `NOT RUN` for every required gate. Missing evidence is not a pass.

2.1 Destructive-action stop points

Explicit approval is required immediately before:

- `helm uninstall`, namespace deletion, PVC deletion, D1/R2/Queue deletion, or DNS removal.
- A database restore using `--clean`, an R2/object-store overwrite, or rollback across a non-backward-compatible migration.
- Production secret rotation that would interrupt clients.
- Store publication, phased release, or tester-group changes.

For each approval, show the account, cluster, namespace, domain, environment, exact object names, backup identifier, and recovery plan.

3. Repository map

Path	Purpose
<code>android/</code>	Android application and modules
<code>ios/TalkApp/</code>	iOS application and UI tests
<code>ios/PttTalk/</code>	Reusable Swift package and protocol/media tests
<code>native/</code>	Shared Rust media and Apple FFI code
<code>server/</code>	Rust control service and UDP relay
<code>admin-web/</code>	TypeScript administrator console
<code>cloudflare/</code>	Edge implementation, migrations, tests, and Wrangler configuration
<code>deploy/helm/ptt/</code>	Supported K3s Helm chart
<code>proto/</code>	Frozen protocol definitions and descriptor
<code>scripts/</code>	Build, release, security, simulator, physical, and deployment gates
<code>docs/</code>	Product, protocol, operations, testing, and security documentation

4. Inputs that must exist before deployment

Create an operator-only work directory outside the checkout. The examples use `/secure/ptt`; choose a protected equivalent.

4.1 Common inputs

- A public HTTPS origin such as `https://ptt.example.com`.
- A second DNS name for K3s gRPC, such as `grpc.ptt.example.com`.
- A TLS certificate covering every public HTTPS/gRPC name.
- A verified SMTP or email-delivery sender.
- A bootstrap email address for the first administrator.
- A Firebase project and least-privilege FCM service-account JSON.
- Two independent Apple Push Notification signing keys restricted to the application topic: one for production and one for sandbox.
- The Apple team ID and bundle ID used by the iOS build.
- Android upload signing material and a Firebase `google-services.json` matching the Android application ID.
- A backup destination with encryption at rest and a tested recovery process.

4.2 K3s inputs

- Linux host(s) running a maintained K3s version satisfying the chart constraint `>=1.28.0-0`.
- A working default storage class for live data and a separately verified encrypted storage class for backups.
- Inbound TCP 443 and UDP 47000 from intended clients.
- Egress to SMTP, FCM, APNs, image registries, and any upstream object-store endpoint.
- `kubectl` context pointing to the intended cluster and Helm 3.
- A container registry to which the operator can push three images.

4.3 Cloudflare inputs

- A Cloudflare account with Workers, D1, R2, Queues, Durable Objects, and Email Service access.
- A zone/domain in that account and a verified Email Service sender.
- Wrangler authenticated to the intended account.
- Unique staging and production resource names and IDs. Never reuse the repository owner's committed Cloudflare IDs in another account.

4.4 Mobile release inputs

- macOS and Xcode for iOS, with Apple Distribution identity, App Store provisioning profile, Push Notifications, Associated Domains, and Apple's managed Push to Talk capability.
- JDK 21, Android SDK API 36, Android NDK, CMake, Rust stable, and `cargo-ndk` for Android.
- The pinned libsignal source at tag `v0.101.0`, including submodules.
- Physical test devices: at least two Android and two iOS devices for the parity gate. Simulator audio is useful but does not prove microphone, route, push wake, or locked-screen behavior.

5. Establish a clean, reproducible baseline

Run from the repository root:

```
git remote -v
git status --short
git fetch --tags --prune origin
git checkout main
git pull --ff-only origin main
git rev-parse HEAD
git describe --always --dirty
```

Expected result: the remote is the intended repository, the checkout is clean, and the selected commit is recorded in the deployment evidence. If deploying a release tag, check out that tag rather than an unreviewed branch.

Record tool versions:

```
git --version
rustc --version
cargo --version
node --version
npm --version
java -version
docker version
helm version
kubectl version --client
protoc --version
```

For Apple builds also record:

```
xcodebuild -version
swift --version
xcrun simctl list runtimes
```

The CI reference uses Node 24, JDK 21, Rust stable, Android compile/target API 36, Android minimum API 26, and iOS minimum version 16. Do not silently substitute an older toolchain.

6. Validate source before packaging

Install JavaScript dependencies with lockfiles:

```
npm ci --prefix admin-web
npm ci --prefix cloudflare
```

Run the portable contract and application gates:

```
./scripts/check-proto-contract.sh
node ./scripts/verify-documentation.mjs
node ./scripts/verify-store-readiness.mjs
cargo fmt --manifest-path native/Cargo.toml --all -- --check
cargo test --manifest-path native/Cargo.toml --locked
cargo fmt --manifest-path server/Cargo.toml --all -- --check
cargo test --manifest-path server/Cargo.toml --locked
./gradlew compileKotlin --no-daemon
./gradlew :crypto:test :floor:test :hardware:test :media:test :loopback:test :net:test
:talkandroid:testDebugUnitTest --no-daemon
./gradlew :crypto-persistence:lintDebug :talkandroid:lintDebug :talkandroid:assembleDebug --no-daemon
(cd ios/PttWire && swift test)
./scripts/test-chat-codec-swift.sh
npm run typecheck --prefix admin-web
npm run build --prefix admin-web
npm run check --prefix cloudflare
```

Then run the disposable server integration suite:

```
./scripts/test-control-integration.sh
```

This suite creates isolated PostgreSQL, Redis, MinIO, push-provider mocks, control, and relay processes. It verifies migrations, health, privacy-safe metrics, invitations, device authentication, prekeys, membership, administrator sessions, push outbox behavior, mailbox idempotency, floor/SOS behavior, authenticated UDP binding, HMAC rejection, source-tuple rejection, replay handling, and cleanup.

Run the two-process encrypted relay fixture:

```
./scripts/two-process.sh
```

Do not continue if any command fails.

7. Build immutable K3s application images

Choose a registry namespace and a tag derived from the commit. This example intentionally uses environment variables instead of fixed operator values:

```
export PTT_REGISTRY=ghcr.io/your-organization
export PTT_COMMIT="$(git rev-parse --short=12 HEAD)"
docker login ghcr.io
```

Build and push multi-platform images. The target K3s nodes must be covered by the selected platforms:

```
docker buildx create --name ptt-release-builder --driver docker-container --use --bootstrap
docker buildx build --platform linux/amd64,linux/arm64 \
  -f server/control/Dockerfile \
  -t "$PTT_REGISTRY/ptt-control:$PTT_COMMIT" --push .
docker buildx build --platform linux/amd64,linux/arm64 \
  -f server/relay/Dockerfile \
  -t "$PTT_REGISTRY/ptt-relay:$PTT_COMMIT" --push .
docker buildx build --platform linux/amd64,linux/arm64 \
  -f admin-web/Dockerfile \
  -t "$PTT_REGISTRY/ptt-admin-web:$PTT_COMMIT" --push .
```

Resolve and record immutable digests:

```
docker buildx imagetools inspect "$PTT_REGISTRY/ptt-control:$PTT_COMMIT"
docker buildx imagetools inspect "$PTT_REGISTRY/ptt-relay:$PTT_COMMIT"
docker buildx imagetools inspect "$PTT_REGISTRY/ptt-admin-web:$PTT_COMMIT"
```

Use the manifest-list [sha256: . . .](#) digest for each image in the production values file. If the registry is private, create a namespace-scoped pull secret and configure the service account or workloads according to the organization's policy before installation.

8. Deploy on K3s with Helm

8.1 Confirm the target

```
kubectl config current-context
kubectl cluster-info
kubectl get nodes -o wide
kubectl get storageclass
helm version
```

Record node architecture, available capacity, ingress class, public IP, storage classes, and backup encryption evidence.

8.2 Configure DNS, TLS, and firewall

Create DNS records for the HTTP/admin host and gRPC host pointing to the K3s ingress address. Confirm them from outside the cluster:

```
dig +short ptt.example.com
dig +short grpc.ptt.example.com
```

Provision a Kubernetes TLS secret in namespace `ptt` whose certificate covers both names. The certificate may come from cert-manager or an operator-controlled issuer. Verify it without exposing the private key:

```
kubectl create namespace ptt --dry-run=client -o yaml | kubectl apply -f -
kubectl -n ptt get secret ptt-tls
kubectl -n ptt get secret ptt-tls -o jsonpath='{.data.tls.crt}' \
  | base64 --decode | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Allow TCP 443 and UDP 47000 at the host firewall, cloud firewall, load balancer, and upstream NAT. Do not expose PostgreSQL, Redis, MinIO, metrics, or cluster-local gRPC directly.

8.3 Create the protected values file

Create `/secure/ptt/operator-values.yaml`, set mode `0600`, and replace every placeholder:


```
publicBaseUrl: https://ptt.example.com

control:
  replicaCount: 1
  image:
    repository: ghcr.io/your-organization/ptt-control
    tag: ignored-when-digest-is-set
    digest: sha256:REPLACE_WITH_CONTROL_MANIFEST_DIGEST
    pullPolicy: IfNotPresent

adminWeb:
  replicaCount: 1
  image:
    repository: ghcr.io/your-organization/ptt-admin-web
    tag: ignored-when-digest-is-set
    digest: sha256:REPLACE_WITH_ADMIN_MANIFEST_DIGEST
    pullPolicy: IfNotPresent

relay:
  replicaCount: 1
  image:
    repository: ghcr.io/your-organization/ptt-relay
    tag: ignored-when-digest-is-set
    digest: sha256:REPLACE_WITH_RELAY_MANIFEST_DIGEST
    pullPolicy: IfNotPresent
  publicAddress: ptt.example.com:47000
  service:
    type: LoadBalancer
    port: 47000

ingress:
  enabled: true
  className: traefik
  host: ptt.example.com
  grpcHost: grpc.ptt.example.com
  tlsSecretName: ptt-tls
  rateLimit:
    enabled: true
    average: 100
    burst: 200
    period: 1s

postgres:
  storage: 10Gi
  storageClassName: local-path

redis:
  storage: 1Gi
  storageClassName: local-path

objectStore:
  bucket: ptt-history
  storage: 20Gi
  storageClassName: local-path

smtp:
  enabled: true
  host: smtp.example.com
  port: 587
  username: ptt@example.com
  from: PTT Talk <ptt@example.com>

push:
  fcm:
    enabled: true
  apns:
    enabled: true
    productionKeyId: REPLACE_WITH_PRODUCTION_KEY_ID
    sandboxKeyId: REPLACE_WITH_SANDBOX_KEY_ID
    teamId: REPLACE_WITH_APPLE_TEAM_ID
    bundleId: app.ptt.talk

backup:
  enabled: true
  schedule: "17 2 * * *"
  storage: 20Gi
  storageClassName: operator-encrypted
  encryptedStorageClassConfirmed: true
  retentionDays: 14
```

```

metrics:
  enabled: true
  serviceMonitor:
    enabled: false
  prometheusRule:
    enabled: false

secrets:
  databasePassword: REPLACE_WITH_UNIQUE_RANDOM_VALUE
  redisPassword: REPLACE_WITH_UNIQUE_RANDOM_VALUE
  objectStorePassword: REPLACE_WITH_UNIQUE_RANDOM_VALUE
  bootstrapToken: REPLACE_WITH_UNIQUE_RANDOM_VALUE_AT_LEAST_32_CHARACTERS
  smtpPassword: REPLACE_WITH_SMTP_PASSWORD
  relaySharedSecret: REPLACE_WITH_UNIQUE_RANDOM_VALUE_AT_LEAST_32_CHARACTERS
  metricsToken: REPLACE_WITH_UNIQUE_RANDOM_VALUE_AT_LEAST_32_CHARACTERS
  fcmServiceAccountJson: '{"type":"service_account","REPLACE":"WITH_COMPLETE_JSON"}'
  apnsProductionPrivateKey: |-
    -----BEGIN PRIVATE KEY-----
    REPLACE_WITH_PRODUCTION_APNS_KEY
    -----END PRIVATE KEY-----
  apnsSandboxPrivateKey: |-
    -----BEGIN PRIVATE KEY-----
    REPLACE_WITH_SANDBOX_APNS_KEY
    -----END PRIVATE KEY-----

```

Important constraints:

- Every password/token placeholder must be unique. Bootstrap, relay, and metrics values must contain at least 32 characters.
- Production and sandbox APNs key IDs and private keys must be independent. The chart rejects reuse.
- `encryptedStorageClassConfirmed: true` is an operator assertion, not encryption. Verify the actual storage implementation and key recovery process.
- Helm stores release values in cluster Secrets. Restrict namespace and cluster access, encrypt Kubernetes data at rest, and never pass the values with `--set` where shell history or process listings may expose them.
- Live `local-path` volumes are node-local. A multi-node production design must use storage with appropriate replication, failure-domain, snapshot, and recovery behavior.

Check for unresolved placeholders without displaying secret values:

```

if grep -Eq 'REPLACE_WITH|your-organization|ptt\.example\.com|operator-encrypted'
  /secure/ptt/operator-values.yaml; then
  echo 'operator-values.yaml still contains placeholders' >&2
  exit 1
fi
chmod 600 /secure/ptt/operator-values.yaml

```

8.4 Render, inspect, and install

```

helm lint deploy/helm/ptt -f /secure/ptt/operator-values.yaml
helm template ptt deploy/helm/ptt \
  --namespace ptt \
  -f /secure/ptt/operator-values.yaml \
  > /secure/ptt/rendered.yaml

```

Inspect the rendered image references, hostnames, storage classes, public services, security contexts, and network policies. Do not publish `rendered.yaml`; it contains Kubernetes Secrets.

Install atomically:

```

helm upgrade --install ptt deploy/helm/ptt \
  --namespace ptt --create-namespace \
  --atomic --wait --timeout 15m \
  -f /secure/ptt/operator-values.yaml

```

Verify Kubernetes state:

```

kubectl -n ptt get pods,deploy,statefulset,service,ingress,pvc,networkpolicy
kubectl -n ptt rollout status deployment/ptt-ptt-control --timeout=5m
kubectl -n ptt rollout status deployment/ptt-ptt-relay --timeout=5m
kubectl -n ptt rollout status deployment/ptt-ptt-admin-web --timeout=5m

```

```
kubectl -n ptt get events --sort-by=.lastTimestamp
```

Expected steady state: all deployments are available, PostgreSQL/Redis/MinIO are ready, PVCs are bound, and no pods are crash-looping or pending.

8.5 External service verification

From a network outside the cluster:

```
curl --fail --silent --show-error https://ptt.example.com/healthz
curl --fail --silent --show-error https://ptt.example.com/readyz
curl --fail --silent --show-error --head https://ptt.example.com/
openssl s_client -connect ptt.example.com:443 -servername ptt.example.com </dev/null
openssl s_client -connect grpc.ptt.example.com:443 -servername grpc.ptt.example.com </dev/null
```

Check that the public site returns HTTPS, the certificate chain and SANs are correct, [/readyz](#) reports ready, HTTP redirects to HTTPS if HTTP is exposed, and security headers are present. Metrics must not be publicly reachable.

Query metrics only through a protected local port-forward:

```
kubectl -n ptt port-forward service/ptt-ptt-metrics 19090:9090
```

In another protected shell, supply the metrics bearer token from the secure values source and request <http://127.0.0.1:19090/metrics>. Verify a request without the token is rejected and that labels do not contain account, device, email, channel, token, key, or media identifiers.

8.6 Bootstrap the first administrator

The bootstrap token is accepted only while no administrator exists. Read it from the protected values source without printing it, then make one request:

```
read -r -p 'Administrator email: ' PTT_ADMIN_EMAIL
read -r -s -p 'Bootstrap token: ' PTT_BOOTSTRAP_TOKEN
printf '\n'
curl --fail --silent --show-error \
  -H 'Content-Type: application/json' \
  --data "$(jq -nc --arg email "$PTT_ADMIN_EMAIL" --arg token "$PTT_BOOTSTRAP_TOKEN" \
    '{email:$email,bootstrapToken:$token}')" \
  https://ptt.example.com/v1/bootstrap
unset PTT_BOOTSTRAP_TOKEN
```

The response contains an invitation code and expiry, and the server queues the email link. Confirm the invitation email arrives, enroll the first device, and verify the account is an administrator. Then rotate the Helm bootstrap value so an old database snapshot cannot reactivate the original token.

Administrator browser access is not a permanent password: in the enrolled app, open **Settings > Open admin console**. The device creates a two-minute, single-use handoff. The browser receives a memory-only session lasting 15 minutes. Sign out when finished.

8.7 Confirm email and push delivery

In the administrator console:

1. Confirm SMTP/email and FCM/APNs production/sandbox show configured.
2. Send an invitation to a controlled test address and confirm the email is received once.
3. Enroll an iOS TestFlight build, an iOS debug/physical build, and an Android build.
4. Confirm production iOS registrations use the production APNs provider and debug physical tests use sandbox.
5. Lock receiving devices and confirm mailbox and voice wake paths.
6. Confirm push payloads contain only the event kind and opaque message UUID, never identity, channel, key, message text, attachment metadata, or audio.

9. Deploy the Cloudflare implementation

9.1 Create an operator overlay

Do not deploy the repository's current production bindings into another account. Copy `cloudflare/wrangler.jsonc` to an operator-controlled branch or overlay and replace:

- Worker names for staging and production.
- D1 database IDs.
- R2 bucket names.
- Email and push queue/dead-letter queue names.
- Production routes and custom domains.
- `PUBLIC_BASE_URL`, `EMAIL_FROM`, `ANDROID_APP_CERT_SHA256`, and `ENVIRONMENT` variables.

Keep the Durable Object binding/class and migration history intact.

9.2 Authenticate and validate locally

```
cd cloudflare
npm ci
npx wrangler whoami
npm run build:admin
npm run check
```

`npm run check` generates Worker types, type-checks, runs Vitest, and performs a staging dry-run deploy.

9.3 Provision staging resources

Use unique names for the operator account:

```
npx wrangler d1 create ptt-talk-staging
npx wrangler r2 bucket create ptt-talk-history-staging
npx wrangler queues create ptt-talk-email-staging
npx wrangler queues create ptt-talk-email-staging-dlq
npx wrangler queues create ptt-talk-push-staging
npx wrangler queues create ptt-talk-push-staging-dlq
```

Put the returned IDs and names into the staging environment of the operator overlay. Apply every migration in order:

```
npx wrangler d1 migrations apply DB --env staging --remote
```

Set secrets interactively; Wrangler must prompt for the value:

```
npx wrangler secret put BOOTSTRAP_TOKEN --env staging
npx wrangler secret put FCM_SERVICE_ACCOUNT_JSON --env staging
npx wrangler secret put APNS_PRODUCTION_KEY_ID --env staging
npx wrangler secret put APNS_PRODUCTION_PRIVATE_KEY --env staging
npx wrangler secret put APNS_SANDBOX_KEY_ID --env staging
npx wrangler secret put APNS_SANDBOX_PRIVATE_KEY --env staging
npx wrangler secret put APNS_TEAM_ID --env staging
npx wrangler secret put APNS_BUNDLE_ID --env staging
```

Production and sandbox APNs credentials must be independent. Configure the Email Service binding and verify the sender domain. Until Email Service is configured, invitation links remain pending and are not written to logs.

Deploy staging:

```
npm run deploy:staging
```

Verify `GET /healthz`, `GET /readyz`, the admin web interface, email delivery, separate APNs readiness, FCM readiness, enrollment, two-client media, attachments, and backup export before creating production resources.

9.4 Provision and deploy production

Repeat resource creation with production-specific names. Apply migrations before traffic:

```
npx wrangler d1 migrations apply DB --env production --remote
npm run deploy:production
```

Immediately verify:

```
curl --fail --silent --show-error https://your-production-domain.example/healthz
curl --fail --silent --show-error https://your-production-domain.example/readyz
```

The health response advertises protocol version, minimum client version, capability set, and structural FCM/APNs readiness. Deploy a server capability advertisement before distributing a client that requires it.

9.5 Cloudflare backup and rollback

Before every schema or production Worker change, export D1 and verify the export in an isolated local SQLite process:

```
npx wrangler d1 export DB --env production --remote --output /secure/ptt/backup.sql
cd ..
./scripts/verify-d1-backup-restore.sh /secure/ptt/backup.sql 0010_collaboration_workspace.sql
```

The verifier checks SQLite integrity, foreign keys, the migration ledger, and required production tables, then removes the temporary restored database. D1 export does not include R2 ciphertext. Maintain a coordinated R2 backup/versioning policy and record an object inventory at the same recovery point. Do not claim backup readiness until both D1 and R2 recovery have been exercised together.

Use Cloudflare deployment versions for Worker rollback. A Worker rollback does not reverse a schema migration. If a migration is not backward compatible, restore the coordinated D1/R2 recovery point under an approved incident procedure.

10. Custom domains, application IDs, and verified links

Users can manually enter another HTTPS server and copy a one-time code, but seamless email and device-link handling requires client rebuilds and domain association.

Before branding a self-hosted build, audit all current fixed identifiers:

```
rg -n 'ptttalk\.app|app\.ptt\.talk|M2M4752Z6K' android ios cloudflare
```

At minimum review and update:

- Android namespace/application ID, manifest app links, device-link URL generation/parsing, release default server, privacy URL, Firebase configuration, signing certificate fingerprint, and matching tests.
- iOS bundle ID, development team, export options, entitlements, universal-link generation/parsing, release default server, privacy URL, Keychain namespaces where identity separation is intended, and matching tests.
- Cloudflare Apple App Site Association response, Android asset-links response, APNs bundle ID, public base URL, privacy/deletion routes, and Android signing-certificate SHA-256.
- Apple Developer and App Store Connect identifiers, Push to Talk approval, Push Notifications, Associated Domains, provisioning profiles, APNs topic restrictions, Google Play record, and Firebase app registration.

Host and verify:

- https://YOUR_DOMAIN/.well-known/apple-app-site-association
- https://YOUR_DOMAIN/.well-known/assetlinks.json

The association files must name the newly signed applications. Test verified links on freshly installed physical devices; existing association caches can hide mistakes.

The K3s control chart does not currently generate these two association responses. Put them on the same public origin using the ingress/front-end layer, or use manual code entry until the responses are implemented and verified.

11. Build and install Android

11.1 Toolchain

Use JDK 21, Android SDK API 36, Android NDK, CMake, Rust stable, and `cargo-ndk`. The release script loads the repository Java 21 environment helper.

11.2 Debug and emulator build

```
./gradlew :talkandroid:lintDebug :talkandroid:assembleDebug --no-daemon
adb install -r android/talk/build/outputs/apk/debug/talkandroid-debug.apk
```

For the Android emulator, the debug default server is `http://10.0.2.2:8080`. A production-like custom server should be entered as HTTPS in the app.

11.3 Signed Play bundle

Keep the upload keystore at `~/.ptt_release/ptt-upload.jks` and configuration at `~/.ptt_release/android.env`, or point `PTT_RELEASE_ENV` to another protected file. Required variables include the keystore path/alias and password retrieval described by `scripts/android-release.sh`. Put the matching Firebase `google-services.json` in the expected local Android location; the repository ignores it.

```
./scripts/android-release.sh
```

Expected outputs:

- `android/talk/build/outputs/bundle/release/talkandroid-release.aab`
- `android/talk/build/outputs/bundle/release/talkandroid-release.aab.sha256`

The script validates Firebase client configuration, runs release lint, builds the AAB, and verifies its JAR signature. Upload only after release gates pass for the exact commit.

12. Build and install iOS

12.1 Fetch pinned libsignal

```
git clone --branch v0.101.0 --recurse-submodules https://github.com/signalapp/libsignal.git
"$HOME/src/libsignal"
```

If the directory already exists, verify its remote, clean status, exact tag, and submodules rather than replacing it.

12.2 Build simulator dependencies and app

```
export LIBSIGNAL_ROOT="$HOME/src/libsignal"
./scripts/build-libsignal-ios-sim.sh
./scripts/build-apple-native.sh
xcodebuild -project ios/TalkApp/TalkApp.xcodeproj -scheme TalkApp \
  -sdk iphonesimulator -destination 'generic/platform=iOS Simulator' \
  -derivedDataPath ios/TalkApp/.derived CODE_SIGNING_ALLOWED=NO \
  LIBRARY_SEARCH_PATHS="$LIBSIGNAL_ROOT/target/aarch64-apple-ios-sim/debug
  native/target/aarch64-apple-ios-sim/release" \
  build
```

Install and probe the simulator app:

```
./scripts/test-ios-simulator-app.sh
ios/TalkApp/.derived/Build/Products/Debug-iphonesimulator/TalkApp.app
```

12.3 Signed TestFlight IPA

Install an App Store provisioning profile containing:

- `aps-environment=production`
- Associated Domains for the selected app-link domain
- Apple's managed Push to Talk entitlement

Ensure an Apple Distribution identity is available. The default profile name is **PTT Talk App Store**; override it with **PTT_IOS_PROFILE**. Set **LIBSIGNAL_ROOT** if it is not at the default path.

```
./scripts/ios-release.sh
```

The script builds native Apple code, archives the app, exports an IPA, verifies the signature and entitlements, confirms the PushToTalk framework and background mode, and writes a SHA-256 file. Do not distribute an IPA if any verification is bypassed.

13. Functional and security test matrix

13.1 Automated local and CI gates

Gate	Command or workflow	Required evidence
Protocol freeze	<code>./scripts/check-proto-contract.sh</code>	Descriptor and generated bindings agree
JVM/Android	Gradle tests and lint in section 6	All modules pass
Rust	Native/server locked tests	Format and tests pass
Swift wire/media	<code>swift test</code> and <code>test-chat-codec-swift.sh</code>	Golden vectors agree
Two-process relay	<code>./scripts/two-process.sh</code>	Encrypted sender/receiver pass
Control integration	<code>./scripts/test-control-integration.sh</code>	Database, relay, push, admin, and abuse cases pass
Cloudflare	<code>npm run check --prefix cloudflare</code>	Types, tests, dry deploy pass
Helm	lint and template with protected values	No render error or placeholder
Clean K3s lifecycle	<code>./scripts/test-k3s-clean-install.sh</code>	Install, backup, restore, upgrade, rollback, restart pass
Security	<code>./scripts/security-audit.sh</code>	Secret scan, critical vulnerability gate, SBOM
Documentation/store	verification scripts in section 6	Links/assets/disclosures synchronized

`test-k3s-clean-install.sh` is destructive only to the disposable k3d cluster it creates. It requires `curl`, Docker, Helm, `jq`, k3d, and `kubect1`. It builds the three application images from the checkout, imports them, installs a fresh two-node test cluster, validates services, writes database and ciphertext-object markers, backs them up, deletes/restores them, exercises upgrade/rollback and node restarts, and removes the disposable cluster.

13.2 Simulator tests

The two-simulator encrypted-media harness needs the same protected test identity material used by physical automation. Create `/secure/ptt/e2e.env` with mode `0600`; never commit or upload it. It must define:

```
PTT_E2E_SERVER=https://production-like.example
PTT_E2E_ACI=REDACTED_ACCOUNT_UUID
PTT_E2E_CHANNEL_ID=REDACTED_CHANNEL_UUID
PTT_E2E_SENDER_MAILBOX=REDACTED_MAILBOX_UUID
PTT_E2E_RECEIVER_MAILBOX=REDACTED_MAILBOX_UUID
PTT_E2E_SENDER_TOKEN=SECRET_DEVICE_TOKEN
PTT_E2E_RECEIVER_TOKEN=SECRET_DEVICE_TOKEN
PTT_E2E_SENDER_IDENTITY_FIXTURE=SECRET_BASE64_JSON
PTT_E2E_RECEIVER_IDENTITY_FIXTURE=SECRET_BASE64_JSON
```

Load the reviewed protected file without echoing values:

```
set -a
. /secure/ptt/e2e.env
set +a
```

Run:

```
./scripts/test-ios-two-simulator-voice.sh
./scripts/test-ios-accessibility.sh
./scripts/test-android-accessibility.sh
```

Also capture store/review surfaces when needed:

```
./scripts/capture-ios-store-screenshots.sh
./scripts/capture-android-store-screenshots.sh
```

Verify onboarding, invitation errors, two-device link flow, Talk/Chat/Activity/Settings navigation, channel selection, repeated press/release, tones, dynamic type/large font, light/dark appearance, rotation/tablet layouts, empty/loading/offline/error states, privacy redaction, and version display.

Simulator success does not satisfy microphone, speaker, Bluetooth, push wake, locked-screen, or Apple system Push to Talk gates.

13.3 Deployed two-client media automation

For Cloudflare or TLS media, provide two real enrolled device tokens and a channel ID through the protected environment above or equivalent CI secret injection:

```
npm run test:deployed-media --prefix cloudflare
```

Never place these tokens in shell history or logs. Prefer a protected CI runner. The probe must confirm authenticated connection, floor grant, encrypted media transfer, receiver acknowledgement, release, and no plaintext fallback.

13.4 Physical-device voice release gates

Use dedicated test accounts and non-production channels. In addition to `/secure/ptt/e2e.env`, provide the physical device identifiers through protected runner configuration:

```
PTT_ANDROID_DEVICE_1=ANDROID_ADB_SERIAL
PTT_ANDROID_DEVICE_2=ANDROID_ADB_SERIAL
PTT_IOS_DEVICE_1=APPLE_DEVICE_IDENTIFIER
PTT_IOS_DEVICE_2=APPLE_DEVICE_IDENTIFIER
```

The two Android serials, two Apple identifiers, sender/receiver mailbox IDs, tokens, and identity fixtures must be distinct where the harness requires it. The debug automation applications must already be installed and trusted. Load the device file the same way as the E2E file, then run:

```
./scripts/test-android-two-physical-voice.sh
./scripts/test-ios-two-physical-voice.sh
./scripts/test-cross-platform-physical-voice.sh
./scripts/test-four-device-parity.sh
./scripts/test-physical-reboot-restoration.sh
```

For external acoustic validation use:

```
python3 ./scripts/analyze-acoustic-tone.py --self-test
PTT_ACOUSTIC_INPUT=AVFOUNDATION_AUDIO_INPUT_INDEX \
./scripts/record-physical-acoustic.sh ./scripts/test-four-device-parity.sh
```

List AVFoundation devices with `ffmpeg -f avfoundation -list_devices true -i ""` and select an external/room microphone capable of hearing every test device. The wrapper records locally, runs the four-device matrix, checks the expected 997 Hz acoustic bursts, and does not upload the temporary room recording.

Required scenarios, in both directions where applicable:

1. Android to Android, iOS to iOS, Android to iOS, and iOS to Android.
2. First press after launch, warm repeated presses, rapid release/repress, and long talk.
3. Receiver foreground, background, screen locked, and push-woken.
4. Wi-Fi, LTE/5G, Wi-Fi-to-cellular transition, UDP available, and UDP blocked/TLS fallback.
5. Speaker, receiver, wired headset, supported Bluetooth route, interruption, and route change.
6. Sender grant/deny/release tones and receiver playback without duplicate Apple system tones.

7. Device reboot/restoration and Android post-reboot tap-to-rearm behavior.
8. Revoked device, removed member, stale epoch, forged grant, replay, tampered frame, and key reuse all fail closed.
9. Chat text, file, voice note, video, resumable upload, interrupted download, history replay, and unauthorized recipient failure.
10. Second-device linking, future-only message access, remote revocation, and admin-approved recovery.

Record timestamps for press, floor grant, first encrypted media, receiver decode, and audible playback. The product goal is warm floor p95 below 150 ms and mouth-to-ear p95 below 400 ms on good Wi-Fi/LTE. Use `scripts/assert-latency-samples.sh` for machine-enforced percentile checks.

The final physical evidence must include audible/acoustic confirmation. A UI label saying "receiving" is not proof of decoded playback.

13.5 Long-duration and lifecycle tests

- Run the eight-hour Android screen-off receive soak workflow on representative Pixel, Samsung, Xiaomi, and Oppo devices.
- Exercise iOS locked-screen Push to Talk restoration, system interruption, force-quit limitations, and one-active-system-channel behavior.
- Fill storage to warning thresholds; confirm graceful refusal and cleanup.
- Interrupt SMTP, APNs, FCM, database, Redis, object storage, UDP, and TLS separately; confirm bounded retries and clear operator health.
- Load-test 64 encrypted channel members and up to 256 connected devices per relay channel.

13.6 Security gate

Install `trivy` and `syft`, then run:

```
./scripts/security-audit.sh
```

This produces dependency/SBOM evidence and fails on secret findings or critical vulnerabilities. Review high findings rather than silently ignoring them. Additionally verify:

- TLS certificate validation and no plaintext downgrade.
- Authorization on every admin, membership, relay, attachment, and mailbox action.
- Single-use/expiry for magic links, device-link codes, relay leases, and admin handoffs.
- Replay and persistent counter recovery.
- Log, metric, crash, and support-export redaction.
- Rate limits for authentication, recovery, floor, uploads, and integrations.
- Network policies and absence of public database/Redis/MinIO/metrics access.
- Independent APNs credentials and least-privilege FCM identity.
- Backup confidentiality, integrity, restoration, retention, and deletion.

An external cryptography review and application penetration test remain required before broad production rollout.

14. Release-gate verification

For a release commit already tested in GitHub Actions, set the repository and exact SHA, then run:

```
export GITHUB_REPOSITORY=golanbenoni/ptt
export GITHUB_SHA="$(git rev-parse HEAD)"
./scripts/verify-release-gates.sh
```

The script requires successful complete CI, bidirectional production voice, four-device physical parity, and Android eight-hour soak workflows for the exact commit, and verifies synchronized Android/iOS version and build numbers. Do not set skip flags for a production release. Internal distribution workflows may defer physical/soak gates only while those dedicated workflows are actively producing the required evidence.

15. Backup, restore, upgrade, and rollback operations

15.1 K3s backup drill

Trigger a backup:

```
kubectll -n ptt create job --from=cronjob/ptt-ptt-backup ptt-backup-manual
kubectll -n ptt wait --for=condition=complete job/ptt-backup-manual --timeout=20m
kubectll -n ptt logs job/ptt-backup-manual --all-containers
```

Verify the backup contains one UTC timestamp with a PostgreSQL custom-format dump and the matching ciphertext-object tree. Test restoration in an isolated cluster before trusting the backup.

For a real restore, obtain explicit approval, stop control and relay writes, snapshot the current volumes, mount the backup PVC only to an operator-controlled temporary pod, restore PostgreSQL with `pg_restore --clean --if-exists`, and mirror the matching object directory into the configured bucket. Redis contains ephemeral floor/presence state and is not restored. Start services and verify readiness, administrator access, devices, channels, ciphertext history, and two-device communication before reopening traffic.

The chart intentionally does not automate destructive restore.

15.2 K3s upgrade

1. Pass all source and deployment gates for the target commit.
2. Build and sign immutable images; record digests and SBOMs.
3. Take and verify a coordinated manual backup.
4. Update only the three application digests in a copy of the protected values file.
5. Render and review a manifest diff without publishing Secrets.
6. Apply atomically:

```
helm upgrade ptt deploy/helm/ptt \
  --namespace ptt --atomic --wait --timeout 15m \
  -f /secure/ptt/operator-values.yaml
helm history ptt --namespace ptt
```

7. Repeat external health, enrollment, two-client voice/chat, push wake, and backup checks.

If health fails and migrations are backward compatible, use `helm rollback ptt REVISION --namespace ptt --wait`. Otherwise restore the coordinated pre-upgrade database and object snapshot before rolling workloads back.

15.3 Secret rotation

Maintain a register for owner, creation time, scope, expiry, last rotation, and recovery path for:

- Database, Redis, object store, relay, metrics, and bootstrap secrets.
- SMTP credential.
- FCM service account.
- APNs production and sandbox keys.
- Android upload signing key and Apple distribution/profile material.
- Cloudflare access and Worker secrets.

Practice rotations in staging. Rotate bootstrap immediately after first-admin enrollment. Rotate a compromised media/control credential as an incident: block transmission or require upgrade; never downgrade encryption.

16. Monitoring and incident readiness

Monitor only aggregate, privacy-safe signals:

- Readiness and dependency health.
- Connected devices and relay connections as counts.
- Floor request latency and denial/timeout counts.
- Push queue backlog, delivery failures, and dead-letter growth.
- Database connection pressure and migration status.
- Object-store capacity, attachment cleanup, and backup age/result.
- Pod restarts, resource saturation, certificate expiry, and storage exhaustion.

Never label metrics with email, ACI, device ID, channel ID, token, key, message ID, attachment name, or media contents.

For a crypto or authorization incident: disable affected transmission paths, preserve privacy-redacted evidence, revoke compromised devices/credentials, rotate affected channel keys, assess historical scope, and require a fixed minimum client version. There is no plaintext emergency mode.

17. Troubleshooting

Symptom	Likely cause	Safe checks and action
Helm renders but pods cannot pull	Example/nonexistent app tag or private registry	Confirm operator-built repositories and immutable digests; verify pull secret
<code>/healthz</code> works but <code>/readyz</code> fails	PostgreSQL, Redis, object store, migrations, or provider dependency	Inspect readiness and pod events/logs with redaction; fix dependency, do not bypass
Invitation says sent but no email arrives	SMTP/Email Service sender not verified or delivery queue failed	Check operations view and privacy-safe queue/DLQ counts; verify sender/domain
App asks to join after joining	Wrong instance/channel, stale membership epoch, or client/server capability mismatch	Refresh channels, compare sanitized server/version info, verify active membership
Floor grant is slow	Cold connection, REST fallback, push wake, or unhealthy relay	Measure each timestamp; keep authenticated media/control connection warm; inspect capability advertisement
Receiver says receiving but no sound	Audio route/session/decode failure	Use acoustic harness, inspect privacy-redacted support report, test speaker/headset route and physical playback graph
iOS microphone route unavailable	PushToTalk/audio session activation or route timing	Verify entitlement/profile and physical route; use current build; collect redacted support report
<code>com.apple.pushtotalk.channel error 5</code>	System channel capability/profile/state mismatch	Verify managed entitlement in signed app, profile refresh, bundle/topic, one active system channel, and reinstall
Verified invitation link opens browser	AASA/assetlinks mismatch or cached association	Verify well-known response, team/package IDs and signing fingerprint; test fresh install
UDP fails	Firewall/NAT/service address	Confirm UDP 47000 externally; verify automatic encrypted TLS fallback; never add plaintext fallback
APNs readiness fails	Missing/reused/wrong-topic keys	Use independent production/sandbox keys and correct team/bundle IDs
Backup gate fails	Unencrypted class, incomplete DB export, missing R2/object snapshot, or stale migration expectation	Stop rollout; correct and rerun full coordinated restore drill

18. Production go/no-go checklist

Every item must be **PASS** before production traffic:

- [] Exact Git commit and synchronized mobile version/build recorded.
- [] Clean source validation, protocol, unit, integration, cloud, Helm, and documentation gates pass.
- [] Three application images are built from that commit, scanned, SBOM-recorded, signed under operator policy, and deployed by immutable digest.
- [] DNS, certificate chain/SANs, HTTPS, gRPC, UDP, and encrypted TLS fallback verified externally.
- [] PostgreSQL, Redis, object store, queues, migrations, network policies, and resource limits healthy.
- [] SMTP/Email Service, FCM, APNs production, and independent APNs sandbox delivery verified.
- [] First administrator enrolled; bootstrap token rotated; admin handoff/session expiry verified.
- [] Two accounts with two devices each pass linking, future-only access, revocation, and recovery.
- [] Android-to-Android, iOS-to-iOS, and cross-platform encrypted voice pass in both directions with audible/acoustic evidence.

- [] Foreground, background, locked-screen, push wake, reboot/restoration, network change, UDP-blocked, and route-change tests pass.
- [] Text, file, voice-note, and video chat pass with interruption/resume and unauthorized-recipient failures.
- [] Latency objectives pass from collected samples.
- [] Eight-hour Android screen-off soak and four-device parity workflow pass for the exact commit.
- [] Coordinated database/object backup and isolated restore drill pass; retention and recovery ownership documented.
- [] Secret scan, vulnerability review, SBOM, log/metric redaction, external crypto review, and penetration test accepted.
- [] Monitoring, alerting, on-call ownership, incident procedures, rollback, and credential rotation are rehearsed.
- [] Privacy policy, deletion route, store disclosures, screenshots, and support documentation match the shipped behavior.

19. AI-agent execution protocol

An AI agent should implement the run as the following state machine:

1. **DISCOVER** - Identify repository, commit, operator, target account/cluster, domain, architecture, and deployment choice.
2. **PREFLIGHT** - Check the safety contract, clean tree, tools, credentials by presence only, DNS plan, storage, and backup target.
3. **VALIDATE_SOURCE** - Run section 6 and capture redacted command status/duration.
4. **BUILD** - Produce images/mobile artifacts from the recorded commit and calculate SHA-256/digests.
5. **REVIEW_CHANGE** - Render infrastructure; summarize non-secret diff, public exposure, storage, migrations, and rollback.
6. **AWAIT_APPROVAL** - Required for the first production apply, destructive actions, restore, credential-impacting rotation, and store publication.
7. **DEPLOY_STAGING** - Apply migrations/resources/workloads to staging and wait for readiness.
8. **VERIFY_STAGING** - Run automated, simulator, two-client, physical, failure-injection, security, and restore tests.
9. **GO_NO_GO** - Produce the checklist with links/hashes to evidence; any **FAIL**, **BLOCKED**, or required **NOT RUN** is **NO-GO**.
10. **DEPLOY_PRODUCTION** - Apply the already-proven immutable artifacts without rebuilding.
11. **VERIFY_PRODUCTION** - Run non-destructive health, version, enrollment, push, and controlled two-client smoke tests.
12. **HANDOFF** - Deliver URLs, commit, image digests, mobile hashes, backup ID, test matrix, residual risks, and rollback command. Never deliver secrets.

19.1 Required evidence manifest

Produce a JSON file like this, with no secrets or personal identifiers:

```
{
  "product": "PTT Talk",
  "version": "0.1.28",
  "build": 31,
  "protocol": "1.1",
  "commit": "FULL_GIT_SHA",
  "deployment": {
    "kind": "k3s-helm-or-cloudflare",
    "environment": "staging-or-production",
    "origin": "https://redacted.example",
    "timestampUtc": "RFC3339",
    "images": {
      "control": "repository@sha256:...",
      "relay": "repository@sha256:...",
      "admin": "repository@sha256:..."
    }
  },
  "artifacts": {
    "androidAabSha256": "...",
```

```
"iosIpaSha256": "...",
"sbomSha256": "...",
"backupId": "NON_SECRET_IDENTIFIER"
},
"gateways": [
  {"name": "protocol", "status": "PASS", "evidence": "path-or-ci-url"},
  {"name": "physical-four-device", "status": "PASS", "evidence": "path-or-ci-url"},
  {"name": "restore-drill", "status": "PASS", "evidence": "path-or-ci-url"}
],
"residualRisks": []
}
```

Validate the manifest as part of handoff. Evidence locations must be access-controlled and privacy-redacted.

20. Related source documents

- [README.md](#) - product overview and repository entry point.
- [docs/CURRENT_STATE.md](#) - current implementation and remaining release gates.
- [docs/PROTOCOL_V1.md](#) and [docs/WIRE.md](#) - cryptographic/control/media contract.
- [docs/SIMULATOR_TESTING.md](#) - detailed simulator and physical-device testing.
- [docs/ADMIN_GUIDE.md](#) - administrator workflows.
- [docs/SECURITY_REVIEW_SCOPE.md](#) - security boundaries and review scope.
- [docs/ANDROID_RELEASE.md](#) and [ios/README.md](#) - platform release details.
- [deploy/helm/ptt/README.md](#) - chart-specific operations.
- [cloudflare/README.md](#) - edge implementation details.

When documentation conflicts with code, stop and reconcile it in a reviewed commit before deployment. Do not guess around a security, storage, migration, entitlement, or protocol mismatch.